

2. B-Tree Structure and Index Maintenance (Rebuild /Reindex)

The document has two independent parts:

- **Part 1 — B-Tree Structure:** how the index is built and why lookups are fast.
- **Part 2 — Index Maintenance:** why indexes degrade and how to rebuild them safely.

Read Part 1 to understand the mechanism. Read Part 2 to operate it in production.

Part 1 — B-Tree Structure

1. Why an index exists

Without an index, PostgreSQL reads every row of the table to answer a query. In a plan this appears as a **Seq Scan**.

With an index, PostgreSQL walks a sorted structure that eliminates most of the data at each step. In a plan this appears as an **Index Scan**.

B-Tree is the default index type:

```
-- These two statements are equivalent:  
CREATE INDEX idx_orders_customer ON orders (customer_id);  
CREATE INDEX idx_orders_customer ON orders USING btree (customer_id);
```

The performance gain is not magic. It is a direct consequence of the structure described below.

2. From sorted array to B-Tree

Scanning an unsorted array costs $O(n)$.

Keeping the array sorted allows binary search: compare the middle element, discard half the range, repeat. That costs $O(\log n)$.

Drawing the sequence of comparisons a binary search performs produces a **binary search tree**: one key per node, smaller values on the left, larger on the right.

That structure is fine in memory but wrong for disk. One key per node makes the tree tall, and each level down is potentially another random read.

A **B-Tree** solves this by storing many keys per node instead of one. More keys means more children, which means a much shorter tree — exactly what you want when every level may be a separate disk page.

The "B" is not "Binary". It is usually read as Balanced; Bayer and McCreight never recorded what they actually intended.

3. Page layout

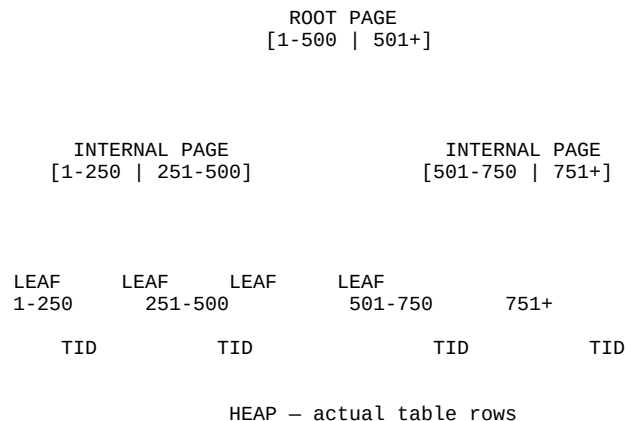
PostgreSQL stores everything in fixed-size pages — 8 KB by default, fixed at compile time. All I/O happens at page granularity, for both the table (heap) and its indexes.

The implementation (nbtree) is a **B+ tree**: only leaf pages point at rows, and internal pages exist purely for navigation.

Apart from a metapage at block 0 that records where the root lives, the index contains three page types:

Level	Page type	Contents
Top	Root	Single entry point; separator keys pointing downward
Middle	Internal	Navigation keys only; no row data
Bottom	Leaf	Index keys + TID pointing to the heap row

A small index has no internal level at all — the root page is itself the only leaf until it fills and splits.



Two consequences matter operationally:

- **Leaf pages form a doubly linked list.** This is what makes BETWEEN, >, < and ORDER BY efficient.
- **The leaf stores a TID, not the row.** PostgreSQL has no clustered index — the heap and every index are separate physical structures. This is the root of most maintenance behaviour in Part 2.

4. Structural properties

Inside a node, keys are sorted ascending. A node with n keys has up to $n + 1$ child pointers, and each key acts as a separator between the child ranges.

The number of children per node is the **fan-out**. Because a page is 8 KB and entries are small, fan-out is in the hundreds.

PostgreSQL guarantees three properties:

1. **Balanced** — every leaf sits at the same depth, so every lookup costs the same number of steps. There are no slow keys.
2. **High fan-out** — hundreds of entries per page keeps depth at roughly 4–5 levels even for very large tables.
3. **Sorted and linked** — data is ordered within and across pages, so an ordered scan is a walk along the leaf level without returning to the root.

An unbalanced binary search tree has a fan-out of 2 and can degenerate into a linked list. A B-Tree forces high fan-out and stays balanced regardless of insert order.

5. How a lookup works

A B-Tree search is neither BFS nor DFS — those would visit most of the tree. It is a **guided descent**: a single path from root to one leaf, choosing exactly one child at each level.

For WHERE `id = 620`:

1. **Root** — binary search the keys: `620 > 500`, follow the right pointer.
2. **Internal** — binary search again: `620` falls in `501-750`, descend.
3. **Leaf** — binary search the leaf, find key `620`, read its TID.
4. **Heap** — follow the TID to read the row and check visibility.

There are two levels of search: descending one branch per level (the tree part), and binary search within each sorted page (the array part).

Step 4 is a separate **heap fetch**. Avoiding it is the reason covering indexes exist:

```
CREATE INDEX idx_orders_customer_covering
ON orders (customer_id)
INCLUDE (status, created_at);
```

If every column the query needs is in the index, PostgreSQL can use an **Index Only Scan**. This skips the heap only for pages marked all-visible in the visibility map; for any other page it still fetches the row to check visibility. INCLUDE requires PostgreSQL 11+.

Range queries work differently: descend once to the first matching leaf, then walk sideways through the linked leaf pages until the upper bound. No return trip to the root. This is what the doubly linked leaf list is for.

6. Why it is fast

Structure	Search	Insert	Delete	Note
Unsorted array / heap	$O(n)$	$O(1)$	$O(n)$	This is a Seq Scan
Sorted array	$O(\log n)$	$O(n)$	$O(n)$	Fast reads, expensive writes
Binary search tree (unbalanced)	$O(n)$ worst	$O(n)$ worst	$O(n)$ worst	Can degenerate to a list
B-Tree	$O(\log n)$	$O(\log n)$	$O(\log n)$	Balanced, disk-friendly

The B-Tree keeps search, insert and delete logarithmic **regardless of insert order** — unlike a plain BST, which degrades to $O(n)$ when values arrive already sorted.

Because the logarithm base is the fan-out (hundreds), not 2, depth stays tiny:

Rows	Tree depth	Page reads per lookup
100	1–2	1–2
10,000	2–3	2–3
1,000,000	3–4	3–4
100,000,000	4–5	4–5

At 100 million rows a lookup touches roughly 5 index pages. A sequential scan over the same table reads on the order of a million pages. That gap is the entire justification for indexes.

In practice the upper levels of a busy index stay resident in `shared_buffers`, so most of those page accesses are memory hits rather than disk reads.

7. Verifying with EXPLAIN

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 620;
```

Output	Meaning
Index Scan using idx_orders_customer	The index is being used
Index Only Scan	Best case — but check <code>Heap Fetches</code> ; a high count means the visibility map is stale
Seq Scan on orders	No usable index, or the predicate is not selective enough
Buffers: shared hit=4	4 pages served from <code>shared_buffers</code> . A read= value counts pages fetched from disk or OS cache

Part 2 — Index Maintenance

8. Why indexes degrade

PostgreSQL uses **MVCC** (Multi-Version Concurrency Control):

- **UPDATE** does not modify a row in place. It writes a new version and marks the old one dead.
- **DELETE** does not remove the row immediately. It marks it dead.

Effectively, every **UPDATE** is a **DELETE** + **INSERT**. Both leave **dead tuples** behind in the heap, and an update normally adds an entry to every index on the table. The exception is a **HOT** update — see Gotchas.

Bloat is the accumulated dead space that has not been reclaimed:

- **Table bloat** — the heap occupies more disk than the live data needs.
- **Index bloat** — the B-Tree carries dead entries and half-empty leaf pages, making it larger and deeper than necessary.

Why it matters:

- More pages read for the same result more I/O.
- A larger index means less of it fits in `shared_buffers`.
- Distorted planner cost estimates, which can cause the index to be ignored entirely.

9. VACUUM vs REINDEX

Command	What it does	Reclaims disk?	Locking
VACUUM / autovacuum	Marks dead space reusable for future writes	Only trailing empty pages	SHARE UPDATE EXCLUSIVE — does not block reads or writes
VACUUM FULL	Rewrites the table compactly, rebuilds all indexes	Yes	ACCESS EXCLUSIVE — blocks everything
REINDEX	Rebuilds one or more indexes from scratch	Yes (index only)	Blocks writes unless CONCURRENTLY

The critical point: regular VACUUM does not compact files. It makes dead space reusable in place. The single exception is that it can truncate completely empty pages at the *end* of a relation — free space in the middle is never returned to the OS. An index can therefore stay structurally bloated even when autovacuum is working perfectly, and that gap is what REINDEX fills.

10. Measuring bloat

PostgreSQL exposes no single "fragmentation %" value. Use two levels of investigation.

Quick — size and usage:

```
SELECT
    schemaname,
    relname      AS table_name,
    indexrelname AS index_name,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size,
    idx_scan     AS times_used
FROM pg_stat_user_indexes
ORDER BY pg_relation_size(indexrelid) DESC
LIMIT 20;
```

`idx_scan = 0` means the index has not been used *since statistics were last reset* — check `stats_reset` in `pg_stat_database` before acting on it, and check replicas too, since each node counts its own reads. Never drop an index backing a PRIMARY KEY or UNIQUE constraint, and be careful with indexes that exist to support a foreign key.

Precise — pgstattuple:

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;

SELECT * FROM pgstatindex('idx_orders_customer');
```

Look at `avg_leaf_density` and `leaf_fragmentation`. A density near 90% is healthy; 50–60% indicates real bloat. Note that `pgstattuple` scans the whole object — avoid running it on large tables during peak hours.

11. REINDEX

PostgreSQL has no "reorganize vs rebuild" choice. REINDEX always rebuilds. The only real decision is **blocking or concurrent**.

Blocking form:

```
REINDEX INDEX idx_orders_customer;  -- one index
REINDEX TABLE orders;             -- all indexes on a table
REINDEX DATABASE my_database;      -- rarely needed
```

This builds a compact B-Tree and swaps it in. It takes a SHARE lock on the parent table, which blocks writes but not reads, plus an ACCESS EXCLUSIVE lock on the index itself — so any query that would have used that index blocks until the rebuild finishes. On a busy production table this is effectively an outage.

Concurrent form (PostgreSQL 12+, use this in production):

```
REINDEX INDEX CONCURRENTLY idx_orders_customer;
REINDEX TABLE CONCURRENTLY orders;
```

It builds the replacement alongside the old index, then swaps and drops the original.

	REINDEX	REINDEX CONCURRENTLY
Blocks writes	Yes	No
Blocks reads	Only queries needing that index	No
Speed	Faster	Slower (more work)

Extra disk	~1× index size	~1× index size
Can leave an invalid index	No	Yes

If interrupted, CONCURRENTLY can leave behind an invalid index, usually suffixed `_ccnew`. There is no data loss and the original index remains usable — find the leftover and drop it:

```
SELECT indexrelid::regclass AS invalid_index
FROM pg_index
WHERE NOT indisvalid;

DROP INDEX CONCURRENTLY idx_orders_customer_ccnew;
```

Manual equivalent (PostgreSQL < 12):

```
CREATE INDEX CONCURRENTLY idx_orders_customer_new ON orders (customer_id);
DROP INDEX CONCURRENTLY idx_orders_customer;
ALTER INDEX idx_orders_customer_new RENAME TO idx_orders_customer;
```

This pattern does not work for an index backing a PRIMARY KEY or UNIQUE constraint — those must be swapped with `ALTER TABLE ... DROP CONSTRAINT` and `ADD CONSTRAINT ... USING INDEX`. Neither `CREATE INDEX CONCURRENTLY` nor `DROP INDEX CONCURRENTLY` can run inside a transaction block.

12. Fillfactor

Fillfactor controls how densely leaf pages are packed at build time. The B-Tree default is 90, leaving 10% free per page.

```
CREATE INDEX idx_orders_customer ON orders (customer_id) WITH (fillfactor = 80);

-- Or change an existing index and rebuild:
ALTER INDEX idx_orders_customer SET (fillfactor = 80);
REINDEX INDEX CONCURRENTLY idx_orders_customer;
```

Be precise about what this buys you. For a B-Tree, fillfactor is applied during the initial build and when the index grows at its right edge (ascending keys). Pages that split later under random inserts are divided roughly in half regardless of the setting, so a lower fillfactor mainly delays splits in the period following a rebuild rather than preventing them permanently.

Lower it for indexes on randomly distributed keys that get rebuilt periodically. For append-only or read-mostly data leave it at 90, or raise it toward 100 for a static index to keep it as compact as possible.

13. Maintenance playbook

1. **Check autovacuum first.** Most "index problems" are autovacuum failing to keep up.
2. **Measure, do not guess.** Check size and `idx_scan`; drop unused indexes; use `pgstattuple` when you need real numbers.
3. **REINDEX only on confirmed bloat.** Always CONCURRENTLY in production; plain REINDEX only in a maintenance window or on small tables.
4. **If bloat returns, fix the cause.** Tune autovacuum for that table, consider a lower fillfactor. REINDEX treats the symptom.

Per-table autovacuum tuning is usually the actual fix:

```
ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.05, -- default 0.2
    autovacuum_vacuum_cost_limit   = 2000  -- allow more work per run
);
```

Check whether it is keeping up via `pg_stat_user_tables` — growing `n_dead_tup` alongside a stale `last_autovacuum` means it is falling behind.

In PostgreSQL, the answer to "my index is bloated" is usually "fix autovacuum," not "REINDEX every night." Scheduled nightly reindexing is a SQL Server habit; here it is a band-aid.

Tooling:

- **pg_repack** — rebuilds tables and indexes with far less locking than `VACUUM FULL`. It still needs a brief `ACCESS EXCLUSIVE` lock at the start and end, a primary key or unique key on the table, and roughly double the disk space. The de-facto community tool.
- **pgstattuple** — precise bloat measurement.
- Managed providers (RDS, Cloud SQL) handle most autovacuum configuration, but per-table overrides still apply.

14. Gotchas

-

VACUUM does not compact files. It can only truncate empty pages at the end of a relation. Reclaiming space from the middle requires VACUUM FULL (locks) or pg_repack (mostly online).

- **UPDATE is expensive on indexed columns.** Each update normally writes new entries into every index. A HOT update avoids that, but only when no indexed column changed *and* the new row version fits on the same heap page — index fewer, smarter columns.
 - **Unused indexes are pure cost.** They slow every INSERT and UPDATE, add work to every VACUUM, and consume cache. DELETE pays indirectly, when vacuum later removes the matching index entries.
 - **Always use CONCURRENTLY in production.** A blocked write queue can take the application down.
 - **REINDEX needs free disk** roughly equal to the index being rebuilt.
 - **On SSDs, physical fragmentation barely matters.** Focus on bloat and cache pressure, not physical ordering.
 - **Check statistics before blaming the index.** Stale planner stats (ANALYZE) cause more "why isn't my index used?" issues than bloat does.
-

Summary

Structure — A B-Tree is a balanced, high-fan-out structure that turns a lookup over 100M rows into roughly 5 page reads. Leaves store a TID pointing into the heap, not the row itself.

Maintenance — MVCC leaves dead tuples behind on every UPDATE and DELETE. VACUUM makes that space reusable but does not shrink files; REINDEX rebuilds a clean B-Tree.

Operations — Use REINDEX CONCURRENTLY in production, and treat recurring bloat as an autovacuum tuning problem rather than a reindexing schedule.

Appendix — FAQ

PostgreSQL has no clustered index, so how is data ordered? It isn't. The table is an unordered heap. CLUSTER table USING index reorders it once, but the ordering is not maintained afterward and the operation locks the table.

When do I get an Index Only Scan? When the index contains every column the query needs (use INCLUDE) and the heap pages are marked all-visible — which VACUUM maintains via the visibility map. Another reason to keep vacuuming healthy.

B-Tree vs other index types? B-Tree is the default and the right choice for equality, ranges and sorting. Alternatives: **GIN** (arrays, JSONB, full-text), **GIST** (geometric, ranges), **BRIN** (very large naturally-ordered tables such as time-series), **Hash** (equality only).